

4th EXERCISE: GREATEST COMMON DIVISOR (GCD) - Euclid's algorithm

The Greatest Common Divisor (GCD) of two numbers is the largest positive number that divides both numbers exactly (with no remainder).

For example, the GCD of 48 and 18 is 6, because:

- 6 divides 48 exactly ($48 \div 6 = 8$)
- 6 divides 18 exactly ($18 \div 6 = 3$)
- No larger number divides both 48 and 18 without leaving a remainder

PROBLEM STATEMENT

This exercise asks us to implement an iterative version of Euclid's algorithm to find the GCD of two integers.

A BIT OF HISTORY

Euclid's algorithm is one of the oldest algorithms still in common use today! It was first described by the ancient Greek mathematician Euclid around 300 BCE in his work "Elements". That makes this algorithm over 2,300 years old - quite impressive for something we still use in modern computing!

HOW EUCLID'S ALGORITHM WORKS

Euclid's algorithm is based on a simple but powerful insight:

If you have two numbers A and B (where A is larger than B), the GCD of A and B is the same as the GCD of B and the remainder when A is divided by B.

Let's understand this with an example before we look at the code:

To find the GCD of 48 and 18:

1. Divide 48 by 18, which gives quotient 2 and remainder 12 ($48 = 18 \times 2 + 12$)
2. Now, instead of finding $\text{GCD}(48, 18)$, we can find $\text{GCD}(18, 12)$
3. Divide 18 by 12, which gives quotient 1 and remainder 6 ($18 = 12 \times 1 + 6$)
4. Now we find $\text{GCD}(12, 6)$
5. Divide 12 by 6, which gives quotient 2 and remainder 0 ($12 = 6 \times 2 + 0$)
6. When the remainder becomes 0, our answer is the divisor, which is 6

So the GCD of 48 and 18 is 6!

ITERATIVE IMPLEMENTATION APPROACH

Euclid's algorithm can be written recursively, but we'll implement it iteratively (using a loop) by following these steps:

1. Start with two positive numbers a and b
2. While b is not zero:
 - Calculate the remainder when a is divided by b ($a \% b$)
 - Replace a with b
 - Replace b with the remainder
3. When b becomes zero, a is the GCD

IMPLEMENTATION IN PYTHON

```
def gcd(a, b):  
    while b != 0:
```

```

    a, b = b, a % b # Replace (a, b) with (b, a % b)
    return a

print(gcd(48, 18)) # Output: 6

```

Python's multiple assignment makes this implementation particularly elegant. It simultaneously updates both variables in a single statement.

IMPLEMENTATION IN C

```

#include <stdio.h>

int gcd(int a, int b) {
    while (b != 0) {
        int r = a % b; // r is the remainder
        a = b;
        b = r;
    }
    return a;
}

int main() {
    printf("%d\n", gcd(48, 18)); // Output: 6
    return 0;
}

```

In C, we need a temporary variable `r` to store the *remainder*, as we can't update both `a` and `b` simultaneously like in Python.

IMPLEMENTATION IN BASH

```

gcd() {
    local a=$1
    local b=$2
    local r

    while [ "$b" -ne 0 ]; do
        r=$((a % b))
        a=$b
        b=$r
    done

    echo "$a"
}

gcd 48 18 # Output: 6

```

Bash follows the same logic but uses its specific syntax for arithmetic operations.

TRACING THROUGH AN EXAMPLE

Let's trace through the execution of `gcd(48, 18)` step by step:

1. Start with `a = 48, b = 18`
2. First iteration:
 - Remainder = $48 \% 18 = 12$ (when 48 is divided by 18)
 - `a` becomes 18
 - `b` becomes 12
3. Second iteration:
 - Remainder = $18 \% 12 = 6$ (when 18 is divided by 12)
 - `a` becomes 12
 - `b` becomes 6
4. Third iteration:
 - Remainder = $12 \% 6 = 0$ (when 12 is divided by 6)
 - `a` becomes 6
 - `b` becomes 0
5. Since `b` is now 0, the loop stops and we return `a = 6` as the GCD

VISUAL STEP-BY-STEP BREAKDOWN

Here's a table showing how the values change in each iteration:

Iteration	a	b	Remainder (a % b)	Action
Start	48	18	-	Begin with a=48, b=18
1	18	12	12	a=b, b=remainder
2	12	6	6	a=b, b=remainder
3	6	0	0	a=b, b=remainder
End	6	0	-	return a=6

A VISUAL WAY TO UNDERSTAND EUCLID'S ALGORITHM

Imagine you have a rectangle with sides of length 48 and 18 units. You want to find the largest square tiles that can fit exactly into this rectangle with no cutting.

1. First, you fit as many 18×18 squares as possible into the 48×18 rectangle. You fit 2 squares, leaving a 12×18 rectangle.
2. Then, you fit as many 12×12 squares as possible into the 12×18 rectangle. You fit 1 square, leaving a 12×6 rectangle.
3. Finally, you fit as many 6×6 squares as possible into the 12×6 rectangle. You fit 2 squares with nothing left over.

The largest square that fits perfectly is 6×6, so the GCD is 6!

WHY THIS ALGORITHM WORKS

The algorithm works because of these mathematical properties:

1. If a number d divides both a and b, it will also divide the remainder when a is divided by b.
2. If a number d divides both b and the remainder when a is divided by b, it will also divide a.

These properties ensure that the GCD of the original numbers is the same as the GCD of the smaller number and the remainder.

EFFICIENCY OF EUCLID'S ALGORITHM

Euclid's algorithm is remarkably efficient:

- **Time Complexity:** $O(\log(\min(a, b)))$. This means the number of steps is proportional to the logarithm of the smaller number, making it very efficient even for large numbers.
- **Space Complexity:** $O(1)$, as we only use a fixed amount of memory regardless of the input size.

This efficiency is one reason why this 2,300-year-old algorithm is still relevant today!

SPECIAL CASES TO CONSIDER

1. **Negative Numbers:** The algorithm works for negative inputs if we take their absolute values, as $GCD(a, b) = GCD(|a|, |b|)$.
2. **Zero as Input:** If one of the inputs is zero, the GCD is the absolute value of the non-zero input. If both are zero, the GCD is mathematically undefined (or sometimes defined as 0).

REAL-WORLD APPLICATIONS

The GCD algorithm has many practical applications:

1. **Simplifying Fractions:** To reduce fractions to their simplest form, divide both the numerator and denominator by their GCD.
2. **Cryptography:** The GCD is used in encryption algorithms like RSA.

3. **Computer Graphics:** Finding the GCD of screen dimensions helps in scaling images properly.
4. **Music Theory:** The GCD helps identify harmonics between different musical frequencies.

RELATED CONCEPT: Least Common Multiple (LCM)

The GCD is closely related to the Least Common Multiple (LCM) through this formula:

$$\text{LCM}(a, b) \times \text{GCD}(a, b) = |a \times b|$$

Once you know the GCD, you can easily find the LCM!

WHAT YOU CAN LEARN FROM THIS EXERCISE

1. **Historical Perspective:** You're implementing one of the oldest known algorithms!
2. **Iterative Programming:** You'll practice using loops to solve problems.
3. **Mathematical Thinking:** You'll see how mathematical concepts translate into code.
4. **Algorithm Efficiency:** You'll understand how a simple algorithm can be remarkably efficient.
5. **Practical Applications:** You'll learn how a basic number theory concept has wide-ranging applications.

CHALLENGE YOURSELF

After mastering the basic algorithm, try these extensions:

1. Modify the function to handle negative numbers correctly
2. Implement a function to find the LCM using the GCD
3. Create a function that simplifies fractions using the GCD

CONCLUSION

Euclid's algorithm for finding the GCD is a beautiful example of computational elegance and efficiency. Despite being over two millennia old, it remains a cornerstone of computational mathematics and provides valuable insights into algorithmic thinking. By mastering this algorithm, you gain a deeper understanding of both mathematics and computer science foundations.